



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری  
نام و نام خانوادگی: علیرضا کریمی



نوت ۲۵ آبان

## فصل چهارم: لایه *Network* بخش *Data – Plane* اسلاید ۱ تا ۲۱ [۱]

در سمت فرستنده، سگمنت‌ها را در دیتاگرام‌ها پوشش و *encapsulate* می‌کنیم و بعد از آن، در سمت گیرنده هم باید سگمنت را به لایه انتقال بدهد.

پروتکل‌های شبکه برخلاف پروتکل‌های لایه انتقال و کاربرد، باید در *host*‌ها و روترهای سر راه پیاده‌سازی شوند. روترهای سر راه، فیلدهای هدر همه دیتاگرام‌های *IP* عبوری را بررسی می‌کند.

*truncated protocol stack*

فقط در *host*‌های مبدا و مقصد، کل پشته پروتکل وجود دارد و در روترهای میانی، پشته تا لایه ۳ (از پایین) وجود دارد که بهش می‌گن *truncated protocol stack* یا «پروتکل استک بریده» می‌گویند.

### توابع لایه شبکه

- تابع اول: *forwarding*: این عمل به‌صورت محلی در هر روتر انجام می‌شود.
- تابع دوم: *routing*: این عمل به‌صورت سرتاسری و همکاری بین روترهاست. برای مسیریابی باید الگوریتم‌های روتینگ داشته باشیم برای محاسبه مسیر از مبدا و مقصد. (برعکس فوروارد که برای یه روتر بود، این از مبدا تا مقصده). بعد طبق این مسیریابی کلی که انجام دادیم، به‌صورت محلی در هر روتر، داده‌ها را فوروارد می‌کنیم.

*data plane*

صفحه داده در مورد همین عمل محلی *forwarding* در هر روتر است؛ یعنی تعیین می‌کند از کدام پورت ورودی به کدام پورت خروجی طبق *forwarding table* منتقل شود. از روی پورت خروجی مندرج در هدر دیتاگرام، تشخیص می‌دهد که به کدام پورت خروجی برود.

*control plane*

صفحه کنترل در سطح کل شبکه است و طبق یه‌سری الگوریتم، مشخص می‌شود که از کدام روترها عبور کنیم. دو نوع پیاده‌سازی این مسیریابی داریم:

- مسیریابی سنتی و پیاده‌سازی توابع مربوطه در روترهای میانی. این روش باعث پیچیدگی پیاده‌سازی روترها می‌شود، به‌خاطر تنوع الگوریتم‌های مسیریابی.
- روش دوم، پیاده‌سازی الگوریتم‌های مسیریابی در نرم‌افزار (موسوم به شبکه‌های *software defined networking* و مبتنی بر نرم‌افزار) است که در یک‌سری سرورهای خاص خارج از روترها پیاده‌سازی می‌شوند.

**روش اول پیاده‌سازی *control plane* (per – router control plane)**

صفحه کنترل در همه روترها وجود دارد. عناصر و مولفه‌های مسیریابی در همه روترها هستن و از طریق همین *control plane* با هم تعامل دارند. توابع روتینگ با هم تعامل دارند و در صفحه کنترل هستند اما جداول فورواردینگ در صفحه داده هستند و به‌طور مستقل در هر روتر وجود دارند.

**روش دوم پیاده‌سازی *control plane* (logically – centralized control plane)**

این پیاده‌سازی در شبکه‌های نرم‌افزار است و به نوعی یک صفحه کنترل مجازا متمرکز است. روترها ساده شدند چون توابع روتینگ از روترها حذف شده و در سرور بیرونی به اسم کنترلر پیاده‌سازی شدند. در هر روتر یک *control agent* وجود دارد و از این طریق با کنترلر در ارتباط است. در واقع گراف مسیریابی در دست کنترلر است و اون جداول فورواردینگ مربوط به مسیریابی را به هر *control agent* می‌دهد.

**مدل سرویس شبکه**

دو مدل سرویس داریم:

نوع سرویس اول: تضمین تحویل یا تضمین تحویل با تامین قید تاخیر.

سرویس دوم: سرویس مورد نیاز برای بسته‌های وابسته به هم *flow*. یعنی *inorder* بودن یا کمینه *bandwidth* یا فاصله بین هر دو بسته متوالی در یک *flow*، قید رعایت فاصله بین هم، موسوم به *inter – packet spacing* را رعایت کنند.

انواع معماری شبکه با ویژگی‌هایشان:

مثلا برای معماری *Internet*، *service model* از نوع *best effort* است و هیچ تضمینی از ترتیب و زمان و کمینه *bandwidth* و این موارد نمی‌دهد. حتی هیچ *feedback* مربوط به *congestion* به *Host* ها نمی‌دهد و مثلا مبدا از

روی رخ دادن *loss* متوجه ازدحام می‌شود. برای معماری *ATM*، انواع *service model* ها وجود دارد که برخی از این موارد مذکور را تضمین می‌دهد.

## معماری روتر

روتر دارای پورت ورودی و چند پورت خروجی است. اتصال پورت‌ها به هم با استفاده از ساختار سویچ سریع *highspeed switching fabric* انجام می‌شود. یک پردازنده در روتر این الگوریتم روتینگ را اجرا می‌کند. الگوریتم‌های روتینگ هر چند میلی ثانیه یک بار اجرا می‌شوند اما عمل فورواردینگ هر چند نانو ثانیه یک بار انجام می‌شود. در ساختار پورت ورودی و خروجی به نحوی است که پروتکل‌های لایه فیزیکی و دیتالینک و شبکه را می‌تواند اجرا کند.

## ساختار پورت ورودی

در بخش مربوط به لایه شبکه، هر پورت عمل *lookup* و *forwarding* را انجام می‌دهد. صف هم داریم. پردازش روتینگ معمولاً جدول فورواردینگ را در هر پورت قرار می‌دهد و طبق آن *match* یا تطبیق می‌کنیم که آدرس مقصد هر بسته مربوط به کدام پورت خروجی است.

بسته به آن پورت منتقل می‌شود که به این *action* می‌گویند. هدف این است که پردازش فورواردینگ در سطح *line speed* باشد؛ یعنی نرخ ورود بیشتر از نرخ پردازش نباشد تا بسته‌ها در صف قرار نگیرند.

- در روش اول روتینگ، این روتینگ طبق آدرس مقصد انجام می‌شود.
- در روش مبتنی بر نرم‌افزار که به آن *generalized forwarding* نیز می‌گویند، از فیلدهای دیگر هدر برای فورواردینگ استفاده می‌شود.

روترها یک دامنه آدرس متوالی (فرضاً مربوط به یک ناحیه جغرافیایی) را با استفاده از *address aggregation* به صورت یک محدوده آدرس واحد در نظر گرفته و برای آن، یک پورت خروجی یکسان اختصاص می‌دهند. قسمت نامشترک از آدرس شروع و آدرس پایان مربوط به یک مدخل را ستاره می‌گذاریم. اگر آدرس مقصد اینگونه با دو مدخل متناظر باشد، به مدخلی با *longest prefix match* منتقل می‌شود. برای بازیابی آدرس با سرعت بالا در جداول فورواردینگ، آنها را با تکنولوژی حافظه *TCAM* پیاده‌سازی می‌کنند. این حافظه در یک سیکل آدرس را *retrieve* می‌کند.

## ساختار سویچ در روتر *switching fabrics*

وظیفه این ساختار، هدایت بسته از بافر پورت ورودی به بافر پورت خروجی است.

سرعت آن با نرخ سوییچ کردن یا فورواردینگ مشخص می‌شود که ضربی از نرخ خط است. دوست داریم برای ورودی  $n$  تایی، نرخ سوییچ کردن،  $n$  برابر سرعت هر خط باشد.

تا در هر  $round$  بسته‌های همه خط‌های ورودی را به خروجی مناسب هدایت کنیم.

سه روش پیاده‌سازی ساختار سوییچ:

- روش اول: به کمک یک کامپیوتر معمولی. بسته ورودی توسط  $cpu$  به حافظه منتقل شده و از آن جا به پورت خروجی منتقل می‌شود. پس ۲ تا  $IO$  است و پهنای باند حافظه محدود و وابسته است.

- روش دوم: استفاده از گذرگاه بورد اصلی کامپیوتر  $bus$ . پورت‌های ورودی و خروجی حافظه مستقل دارند. بسته‌ها از طریق باس از حافظه پورت ورودی به پورت خروجی می‌روند و در هر لحظه فقط بسته یک پورت ورودی به پورت خروجی می‌رود. اگر بسته‌های پورت‌های دیگر بخواهند منتقل شوند، به مشکل  $bus contention$  می‌خوریم. پس پهنای باند باس، سوییچینگ را محدود می‌کند.

- روش سوم: استفاده از شبکه‌های میان‌ارتباطی در روترها. پورت‌های ورودی در سطر و پورت‌های خروجی در ستون هستند و می‌توان حداکثر تا  $n$  ورودی را به  $n$  خروجی به‌طور همزمان وصل کرد؛ مشروط بر اینکه هر ورودی با یک خروجی مجزا ارتباط بگیرد.

بهینه‌سازی‌ها: شکستن بسته‌ها به سلول‌هایی و منتقل کردن آنها: موسوم به  $cell switching$

## مراجع

[1] “ch4-v8-slide-1-to-21,” <https://drive.google.com/drive/folders/>, 2022.